



UNIVERSIDADE ESTADUAL DE MATO GROSSO DO SUL – UEMS
CURSO DE CIÊNCIA DA COMPUTAÇÃO

CAIO EDUARDO GOUVEIA DIAS
RGM 47535

ALGORITMO E ESTRUTURA DE DADOS II
RELATÓRIO DOS ALGORITMOS DE ORDENAÇÃO

DOURADOS-MS
2024

INTRODUÇÃO:

A proposta desenvolvida visou a implementação de determinados algoritmos de ordenação, previamente vistos em aula, sendo eles:

- Bubble-sort original;
- Bubble-sort melhorado;
- Insertion-sort;
- Mergesort;
- Quicksort com pivô sendo o último elemento;
- Quicksort com pivô sendo um elemento aleatório;
- Quicksort com pivô sendo a mediana de três;
- Heapsort.

Cada versão desenvolvida foi testada com três tipos de entrada: em ordem crescente, decrescente e aleatória. E recebeu 5 tamanhos de entrada de dados em seus testes: **10, 100, 1.000, 10.000 e 100.000**, além disso os algoritmos *Mergesort*, *Quicksort com pivô sendo um elemento aleatório*, *Quicksort com pivô sendo a mediana de três* e *Heapsort* receberam também os tamanhos de entrada de **500.000, 5.000.000 e 50.000.000**.

Para o desenvolvimento do presente relatório o programa desenvolvido foi testado no Laboratório de Computação 4, seguindo as instruções previamente apresentadas.

METODOLOGIA:

O programa testado recebe de entrada um arquivo binário, gerado pelo programa gera-in-out.c, que contém uma sequência de números inteiros. Após a ordenação, a nova sequência é armazenada em um novo arquivo binário.

A entrada consiste em 3 argumentos:

- 1° O número do algoritmo escolhido (1 a 8);
- 2° O nome do arquivo de entrada;
- 3° O nome do arquivo de saída.

Após o desenvolvimento todos os algoritmos foram testados, com os casos anteriormente citados, de forma continua no Laboratório de Computação 4 e os tempos de execução foram registrados afim do desenvolvimento dos resultados presentes no relatório, os resultados de tempo são apresentados em segundos e possuem fixação de 3 casas decimais.

No Quicksort foram desenvolvidas duas funções e uma delas tinha o objetivo de definir o pivô a ser utilizado.

DESENVOLVIMENTO:

Após os testes os seguintes resultados, em relação ao tempo de execução, foram encontrados:

1) Bubble-sort original:

/	CRESCENTE	DECRESCENTE	ALEATÓRIO
10	0	0	0
100	0	0	0
1.000	0,003	0,006	0,005
10.000	0,0249	0,466	0,446
100.000	23,980	45,753	49,963

2) Bubble-sort melhorado:

/	CRESCENTE	DECRESCENTE	ALEATÓRIO
10	0	0	0
100	0	0	0
1.000	0	0,005	0,003
10.000	0	0,371	0,358
100.000	0	36,185	37,215

3) Insertion-sort:

/	CRESCENTE	DECRESCENTE	ALEATÓRIO
10	0	0	0
100	0	0	0
1.000	0	0,002	0,001
10.000	0	0,159	0,083
100.000	0,001	14,923	7,448

4) Mergesort:

/	CRESCENTE	DECRESCENTE	ALEATÓRIO
10	0	0	0
100	0	0	0
1.000	0	0	0
10.000	0,002	0,002	0,002
100.000	0,015	0,014	0,023
500.000	0,076	0,072	0,117
5.000.000	0,762	0,755	1,265
50.000.000	8,562	14,703	14,703

5) Quicksort com pivô sendo o último elemento:

/	CRESCENTE	DECRESCENTE	ALEATÓRIO
10	0	0	0
100	0	0	0
1.000	0,002	0,002	0
10.000	0,135	0,136	0,002
100.000	12,755	12,758	0,015

6) Quicksort com pivô sendo um elemento aleatório:

/	CRESCENTE	DECRESCENTE	ALEATÓRIO
10	0	0	0
100	0	0	0
1.000	0	0	0
10.000	0,001	0,001	0,002
100.000	0,009	0,009	0,017
500.000	0,047	0,049	0,084
5.000.000	0,505	0,507	0,937
50.000.000	5,542	5,752	10,545

7) Quicksort com pivô sendo a mediana de três:

/	CRESCENTE	DECRESCENTE	ALEATÓRIO
10	0	0	0
100	0	0	0
1.000	0	0	0
10.000	0,001	0,038	0,002
100.000	0,006	3,200	0,015
500.000	0,030	63,320	0,079
5.000.000	0,324	/	0,878
50.000.000	3,492	/	9,995

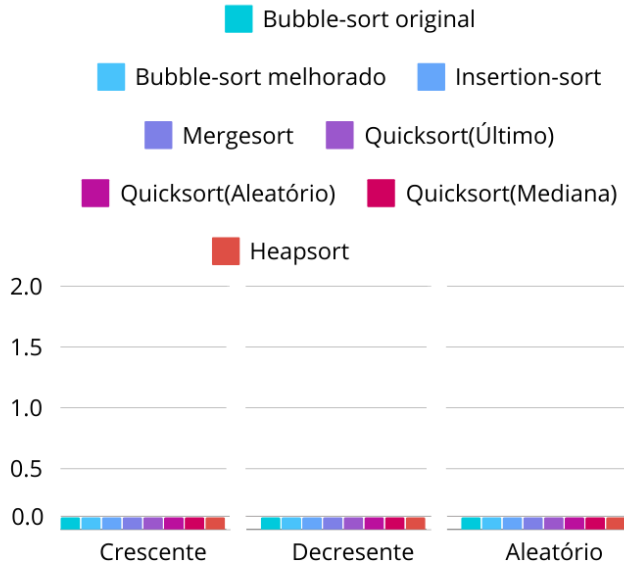
8) Heapsort:

/	CRESCENTE	DECRESCENTE	ALEATÓRIO
10	0	0	0
100	0	0	0
1.000	0	0	0
10.000	0,003	0,003	0,004
100.000	0,030	0,029	0,037
500.000	0,152	0,148	0,186
5.000.000	1,729	1,655	2,485
50.000.000	19,540	18,879	36,485

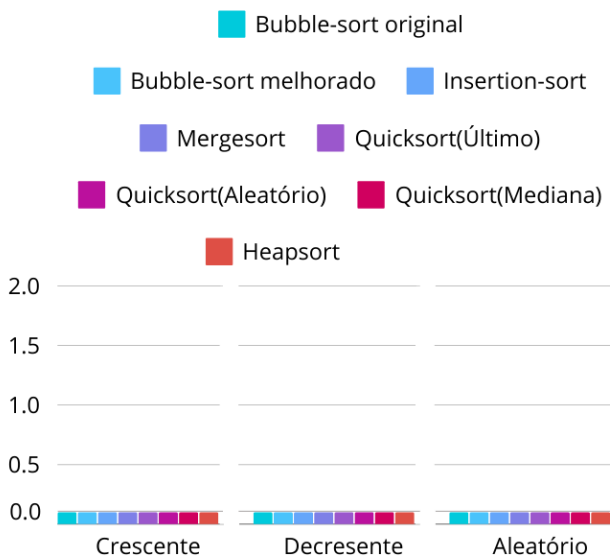
É possível estabelecer uma melhor visualização em relação aos algoritmos mais indicados, em relação a velocidade, por meio dos seguintes gráficos que estão divididos pelos diferentes tamanhos de entrada.

Nota-se que, quanto menor o tamanho da coluna, mais rápido o programa foi capaz de realizar, ressalta-se ainda, que em algoritmos muito discrepantes não será possível identificar colunas em valores 0 ou muito próximos de 0.

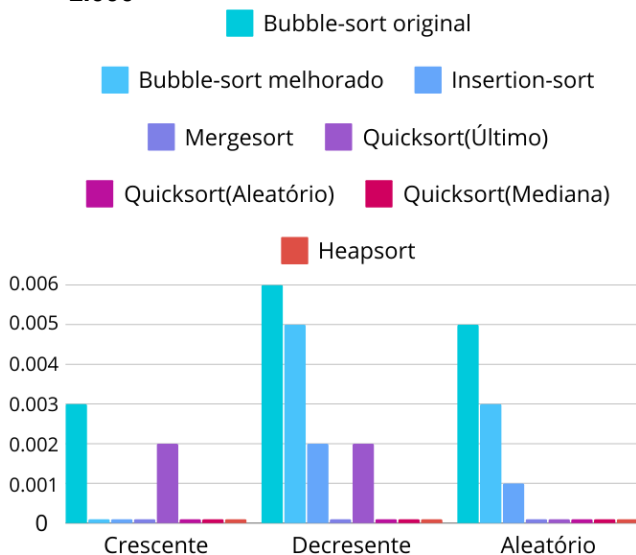
• 10:



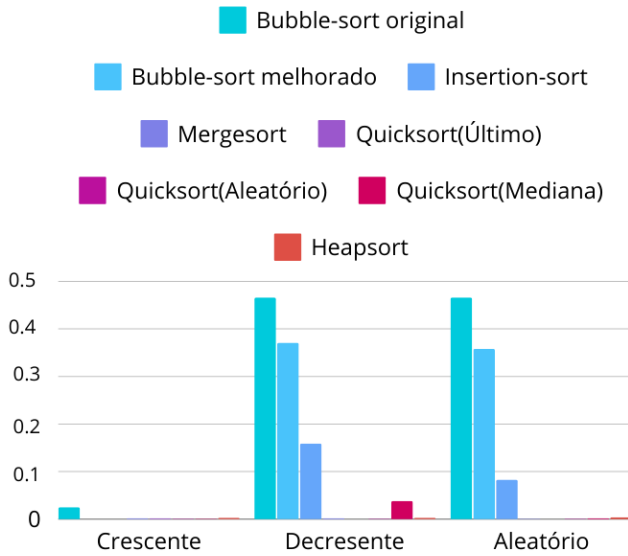
• 100



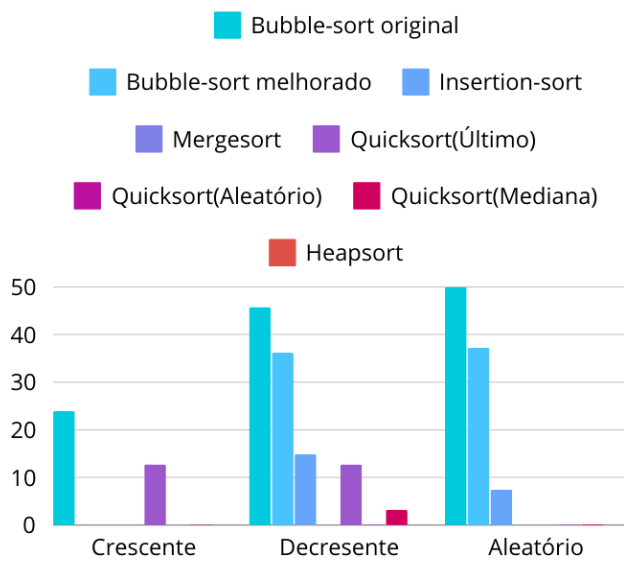
• 1.000



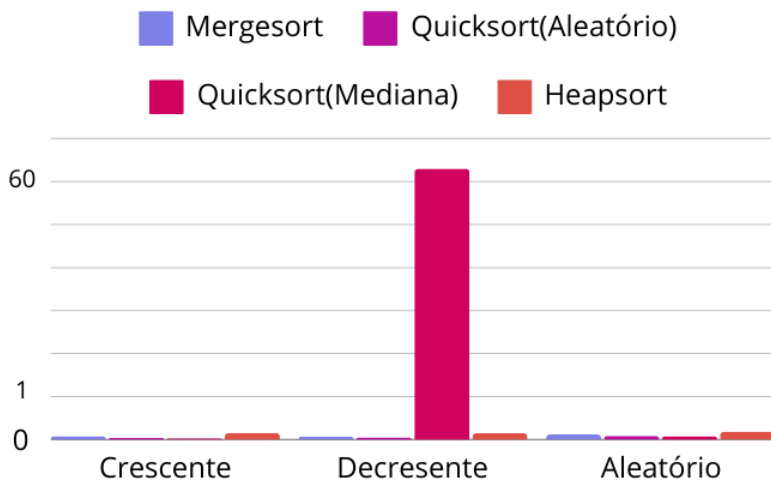
• 10.000



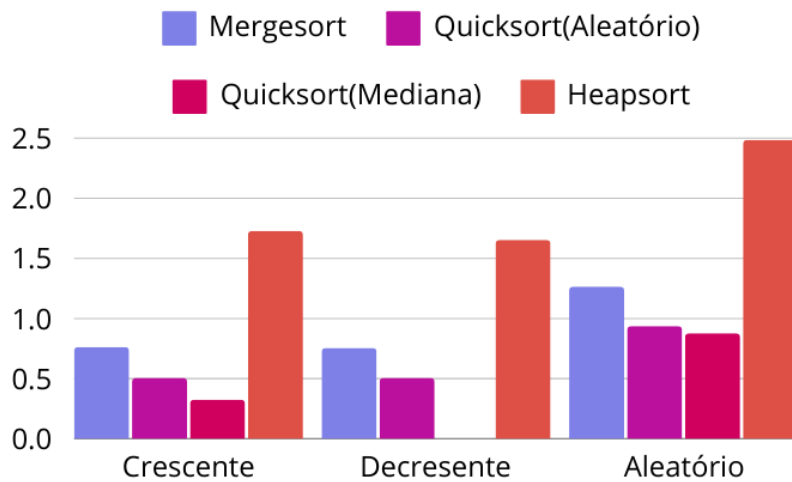
• 100.000



• 500.000

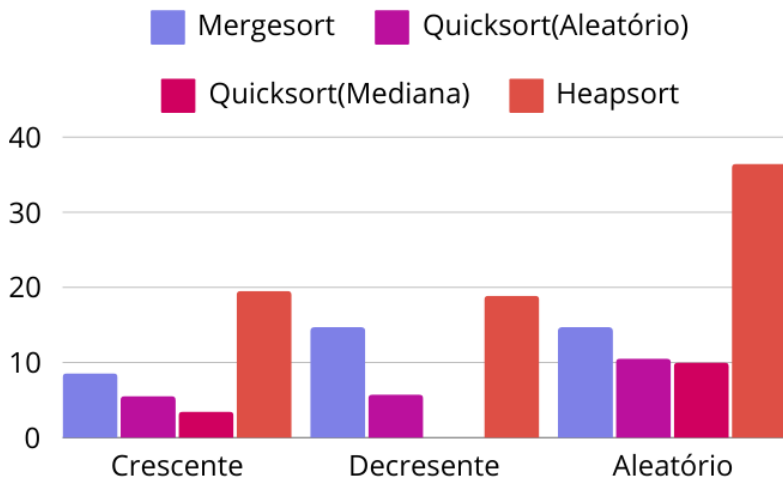


- 5.000.000



OBS: O Quicksort Mediana de 3 não conseguiu finalizar no modo decrescente.

- 50.000.000



OBS: O Quicksort Mediana de 3 não conseguiu finalizar no modo decrescente.

CONCLUSÕES FINAIS:

É notório, como nas amostras de pequeno porte (10 e 100), quaisquer algoritmos apresentam o mesmo resultado nulo, evidenciando como a efetividade é relativa. Os algoritmos se comportam de maneira diferente em casos de amostras diferentes e tipos de ordenação (crescente, decrescente e aleatório), e cada um indica maior afinidade para determinados tamanhos amostrais nos casos de teste.

Entretanto, ainda é possível notar efeitos destoantes entre alguns algoritmos. Para os programas que receberam apenas as amostras de menor fluxo (até 100.000), é visível como o *Bubble-sort original* comporta-se de maneira menos eficaz e mais lenta com média geral de 39.899 de execução em seu caso de maior fluxo, já o *Insertion-sort*, por sua vez, apresenta a maior eficácia nesses pequenos casos, com uma média geral de 7.456 segundos de execução em seu caso de maior fluxo. Na ordenação crescente há uma diferença de 23.979 segundos entre esses algoritmos, no caso decrescente é encontrado uma diferença de 30.830 segundos, e no caso aleatório a diferença chega a 42.512 segundos. Porém, é de interesse ressaltar, que nesse último caso o *Insertion-sort* nem se mostra o algoritmo mais eficaz, uma vez que, *Quicksort com pivô sendo o último elemento*, se mostra mais eficiente, com uma diferença de 7.433 segundos entre ele e o *Insertion-sort* e 49.948 segundos entre ele e o *Bubble-sort original*, fortalecendo o argumento anteriormente apresentado sobre a volatilidade da eficiência e velocidade.

Nos algoritmos que receberam a maior sequência de valores (50.000.000) é possível encontrar uma distância nos algoritmos *Quicksort com pivô sendo um elemento aleatório* e o *Quicksort com pivô sendo a mediana de três*. Entretanto, a parte mais interessante nesses casos, é que diferente dos cenários anteriores em que o melhor caso apresentava maior eficácia em todos os atributos, aqui o protótipo *Quicksort com pivô sendo a mediana de três* é superior ao *Quicksort com pivô sendo um elemento aleatório* nos casos crescente e aleatórios, com uma diferença de 2.050 segundos e 0.550 segundos, respectivamente, porém nos casos decrescentes a discrepância é muito evidente, uma vez que, enquanto o *Quicksort com pivô sendo um elemento aleatório* realiza o programa em 0.775 segundos para um fluxo de 5.000.000 elementos, e em 14.703 segundos para 50.000.000, o *Quicksort com pivô sendo a mediana de três* apresenta uma demora de cerca de 600 a 900 segundos (10 a 15 minutos) para o primeiro caso e 3600 a 5400 segundos (1 hora a 1 hora e meia) para o segundo caso, o que torna as médias gerais extremamente destoantes.

Por fim, nota-se, de modo geral, que os casos decrescentes aprestam os maiores tempos de execução e os crescentes os menores. E diferentes algoritmos comportam-se de maneiras diferentes para os casos de teste.